

messages: 2

You

action = [share1,share2,cash] share_rate 範圍落在0~1，表示資產配置的比例 對action 做softmax 使總值保持在1 new_action = softmax(action) new_action = [share_rate1,share_rate2,cash_rate]

```
portfolio_value += cash
```

$$\text{old_cash_rate} = \text{cash} / \text{portfolio_value}$$
$$\text{delta_cash} = \text{cash_rate} - \text{old_cash_rate}$$

```
for i in range(len(shares)): trade_value[i] = delta_share[i] * portfolio_value
```

重置old value old value = value "" 資料是股市的OHLCV，根據

針對你目前設計的環境邏輯，我有幾個觀察和建議，可以作為你繼續往下建構時的參考：

1. 動作空間（Action Space）與演算法的適配

你使用了 Softmax 將 action 轉換為加總為 1 的資產配置比例 ([share_rate1, share_rate2, cash_rate])，這是一個處理連續動作空間（Continuous Action Space）的絕佳方法。

- 連續型演算法：這樣的設計非常適合 DDPG、SAC 或 PPO 等策略梯度（Policy Gradient）演算法。
- 離散型演算法考量：如果你的研究主軸是評估基於 Value-based 的架構（例如傳統 DQN 或引入注意力機制的 Q-Transformer），這類演算法原生預期的是離散動作空間（Discrete Action Space）。在這種情況下，你可能需要將連續的比例進行離散化（例如：將可選權重切分為 0%, 25%, 50%, 75%, 100% 的組合），或是調整網路輸出層以兼容這種連續資產分配。

2. 手續費（Transaction Costs）的設計

你在買賣時都扣除了 0.003% (0.3%) 的手續費，這點非常貼近真實市場（涵蓋了手續費與潛在的滑價）。

- 目前的邏輯 `fee = abs(trade_value) * transaction_rate` 會有效地在訓練過程中懲罰模型過度頻繁的交易（Churning）。這能迫使 Agent 放棄微小的短期波動，轉而學習更具長遠視野（Long-horizon）的佈局策略。

3. 獎勵函數（Reward Function）

你的 Reward 定義為單步收益率（Rate of Return）： $Reward = \frac{Value - Old_Value}{Old_Value}$ 。

- 這是最直觀且能直接反映資產增長的指標。如果在實驗中發現模型為了追求高報酬而承擔了極端的波動風險，後續可以考慮將 Reward 調整為風險調整後報酬（例如單步的夏普比率 Sharpe Ratio），或者在 Reward 中加入對波動率的懲罰項。

既然「根據...」後面的內容斷掉了，請問你接下來希望根據這份 OHLCV 資料與環境設定完成什麼具體任務？是要將這些邏輯完整封裝成 OpenAI Gymnasium 的 `step()` 與 `reset()` 函數，還是需要針對你預計使用的特定網路架構來進行 State 的特徵工程（Feature Engineering）？